Diet Plan Recommentadion System For Health Issues Using Machine Learning
UCSC398J

Submitted in partial fulfillment of the requirements for the degree of

Bachelor of Science

in

Computer Science

by

HARIPRIYA R

23BCS0124

Under the guidance of

Dr. ANGULAKSHMI M

School of Computer Science Engineering and Information Systems

VIT, Vellore

April, 2026

DECLARATION

I hereby declare that the UCSC398J – Project thesis entitled **"Diet Plan Recommendation System for Health Issues Using Machine Learning"** submitted by me, for the award of the degree of ***Bachelor of Science in Computer Science, School of Computer Science Engineering and Information Systems*** to VIT is a record of bonafide work carried out by me under the supervision **Dr. ANGULAKSHMI M , SCORE, VIT, Vellore.**

I further declare that the work reported in this thesis has not been submitted and will not be submitted, either in part or in full, for the award of any other degree ordiploma in this institute or any other institute or university.

Place: Vellore

Date:

Signature of the Candidate

CERTIFICATE

This is to certify that the UCSC398J – Project thesis entitled **"Diet Plan Recommendation System for Health Issues Using Machine Learning"** submitted by HARIPRIYA R & 23BCS0124, SCORE, VIT, for the award of the degree of ***Bachelor of Science in Computer Science, School of Computer Science Engineering and Information Systems***, is a record of bonafide work carried out by her under my supervision during the period, 05. 12. 2025 to 17.
04. 2026, as per the VIT code of academic and research ethics.

The contents of this report have not been submitted and will not be submitted either in part or in full, for the award of any other degree or diploma in this institute or any other institute or university. The thesis fulfills the requirements and regulations of the University and in my opinion meets the necessary standards for submission.

Place: Vellore
Date:

Signature of the Guide

Internal Examiner **External Examiner**

Head of the Department
Department of Computer Applications

ACKNOWLEDGEMENT

It is my pleasure to express with a deep sense of gratitude to my **UCSC398J - Project** guide **Dr. ANGULAKSHMI M** School of Computer Science Engineering and Information Systems, Vellore Institute of Technology, Vellore for **her** constant guidance, continual encouragement, in my endeavor. My association with **her** is not confined to academics only, but it is a great opportunity on my part to work with an intellectual and an expert in the field of **MACHINE LEARNING.**

"I would like to express my heartfelt gratitude to Honorable Chancellor **Dr. G Viswanathan**; respected Vice Presidents **Mr. Sankar Viswanathan**, **Dr. Sekar Viswanathan**, Vice Chancellor **Dr. V. S. Kanchana Bhaaskaran**; Pro-Vice Chancellor **Dr. Partha Sharathi Mallick**, and Registrar **Dr. Jayabarathi T**.

My whole-hearted thanks to Dean **Dr. Daphne Lopez,** School of Computer Science Engineering and Information Systems, Head, Department of Computer Applications, **Dr. E.Vijayan**, **Bachelor of Science in Computer Science** Project Coordinator **Dr. Jayaram Reddy A**, SCORE School Project Coordinator **Dr. Thandeeswaran R**, all faculty, staff and members working as limbs of our university for their continuous guidance throughout my course of study in unlimited ways

It is indeed a pleasure to thank my parents and friends who persuaded and encouraged me to take up and complete my project successfully. Last, but not least, I express my gratitude and appreciation to all those who have helped me directly or indirectly towards the successful completion of the project.

Place: Vellore

Date: **HARIPRIYA R**

Executive Summary

NutriMind is a web-based Diet Plan Recommendation System that uses Machine Learning to provide personalized dietary guidance for individuals with specific health conditions. The system collects the user's health profile including age, BMI, blood pressure, blood sugar, activity level, smoking, and alcohol habits through a structured web form. A trained Random Forest Classifier model then predicts the user's health condition from six categories — Diabetes, Hypertension, Obesity, Heart Disease, Thyroid, and Normal — and generates a personalized 30-day meal plan with Breakfast, Lunch, Snacks, and Dinner for each day.

The system is built using Python Flask for the backend API, MySQL for relational database management, and HTML, CSS, JavaScript for the frontend. Three Machine Learning algorithms — Random Forest, Support Vector Machine (SVM), and Gradient Boosting — were trained and compared on a merged dataset of 18,000 real patient records from three Kaggle datasets. The Random Forest model achieved the best Test Accuracy of 99.50% with a 5-Fold Cross-Validation accuracy of 99.56%. The system allows users to securely register, log in, submit health data, view their personalized diet plan, and download it as a PDF report.

CONTENTS

		Page No.
Acknowledgement		iv
Executive Summary		v
Table of Contents		vi
List of Figures		ix
List of Tables		x
Abbreviations		xi
Symbols and Notations		xii
1	**INTRODUCTION**	1
	1.1 Problem Objective	1
	1.2 Motivation	2
	1.3 Background	2
	1.4 Literature Survey and Research Synthesis	3
2	**DISSERTATION / INTERNSHIP DESCRIPTION AND GOALS**	6
	2.1 Problem Statement and Context	6
	2.2 Project Vision	6
	2.3 Proposed Solution	6
	2.4 Functional Requirements	7
	2.5 Non-Functional Requirements	8
	2.6 User Person Analysis	8
	2.7 Key Performance Indicators	8

3 **TECHNICAL SPECIFICATION** 9

3.1 System Architecture 9

3.2 Frontend Layer 10

3.3 Backend Layer (Python Flask) 10

3.4 Machine Learning Layer 11

3.5 Database Layer (MySQL) 11

3.6 Technical Stack 12

4 **DESIGN APPROACH AND DETAILS** 13

4.1 Design Approach 13

4.2 Dataset Design 13

4.3 Feature Engineering 13

4.4 Model Selection Rationale 14

4.5 Security Design 14

4.6 Codes and Standards 15

4.7 Constraints and Tradeoffs 15

5 **SCHEDULE, TASKS AND MILESTONES** **16**

5.1 Project Methodology 16

5.2 Phase-wise Task Breakdown 16

5.3 Project Timeline 17

6 **DISSERTATION / INTERNSHIP DEMONSTRATION** **18**

6.1 Demonstration Overview 18

6.2 Step-by-Step Feature Walkthrough 19

6.3 Scenario Walkthroughs 20

7 **COST ANALYSIS / RESULT & DISCUSSION** **21**

7.1 Economic Analysis 21

7.2 Quantitative Results 21

7.3 Cross Validation Analysis 24

7.4 Discussion 24

7.5 Advantages 24

7.6 Limitations 26

8 **SYSTEM IMPLEMENTATION AND DEMONSTRATION** **26**

8.1 Technical Implementation Architecture 26

8.2 System Modules 26

8.3 Database Implementation 33

8.4 Security Implementation 35

8.5 System Demonstration 34

8.6 System Performance 37

8.7 Overcoming Implementation Challenges 37

9 **SUMMARY** **39**

9.1 Summary of Findings 39

9.2 Recommendations 39

9.3 Future Work 39

10 **REFERENCES** **41**

APPENDIX A **42**

List of Figures

Figure No.	Title	Page No.
2.1	Database Entity Relationship Diagram — Three Tables with Foreign Keys	7
3.1	HNutriMind System Architecture — Three-Tier Design	9
3.2	Dataset Distribution — 18,000 Balanced Records (3,000 per condition)	10
4.1	Random Forest — Feature Importance Rankings	14
6.1	NutriMind Complete User Flow Diagram	18
7.1	Classification Report — Precision and Recall per Health Condition	23
7.2	Accuracy	23
8.1	Sign In Page	27
8.2	Sign up Page	27
8.3	Form Page (a)	29
8.4	Form Page (b)	29
8.5	Form Page (c)	29
8.6	ML Prediction Flow — Flask /predict Route	31
8.7	Result Page	32
8.8	PDF Generation Module	33
8.9	User Table	33
8.10	health_formsTable	34
8.11	diet_results Table	34

List of Tables

Table No.	Title	Page No.
3.1	Technical Stack & Versions	12
5.1	Project Timeline	17
7.1	Algorithm Comparison Results	21
7.2	Classification Report — Random Forest	22
7.3	Feature Importance Rankings — Random Forest	22
8.1	System Performance Metrics	37

List of Abbreviations

ML	-	Machine Learning
RF	-	Random Forest
SVM	-	Support Vector Machine
CV	-	Cross Validation
BMI	-	Body Mass Index
API	-	Application Programming Interface
HTML	-	HyperText Markup Language
CSS	-	Cascading Style Sheets
JS	-	JavaScript
DB	-	Database
SQL	-	Structured Query Language
JSON	-	JavaScript Object Notation
PDF	-	Portable Document Format
BP	-	Blood Pressure
CORS	-	Cross-Origin Resource Sharing
UI	-	User Interface
URL	-	Uniform Resource Locator
HTTP	-	HyperText Transfer Protocol

SYMBOLS AND NOTATIONS

Symbol	Description
n_estimators	Number of decision trees in Random Forest
X_train	Maximum depth of each decision tree
X_test	Training feature matrix
y_train	Testing feature matrix
y_test	Training labels
α (alpha)	Learning rate
k	Number of folds in Cross Validation
R^2	Coefficient of determination
%	Percentage accuracy
σ (sigma)	Standard deviation of CV accuracy
f1	F1-Score (harmonic mean of precision and recall)

CHAPTER 1

INTRODUCTION

1.1 Problem Objective:

The primary objective of this project is to design, develop, and deploy a Machine Learning-powered Diet Plan Recommendation System that:

- Collects a structured health profile from the user including age, height, weight, BMI, blood pressure, blood glucose, water intake, activity level, smoking habits, and alcohol consumption.
- Predicts the user's primary health condition from six categories using a trained Random Forest Classifier model.
- Generates a personalized 30-day meal plan with four meals per day — Breakfast, Lunch, Snacks, and Dinner — based on the predicted condition.
- Stores user data securely in a normalized MySQL relational database with three tables: users, health_forms, and diet_results.
- Provides a downloadable PDF report of the complete 30-day diet plan.
- Allows users to securely register and log in using bcrypt password hashing.

The sub-objectives of this project are:

1. To compare three Machine Learning classification algorithms — Random Forest, Support Vector Machine, and Gradient Boosting — on the same dataset and select the best performing model.

2. To build a full-stack web application using Python Flask, MySQL, HTML, CSS, and JavaScript with no external frontend framework dependency.

3. To achieve a model accuracy above 90% using real patient data from publicly available Kaggle datasets.

1.2 Motivation :

The motivation for this project arises from three key observations about the current state of dietary healthcare.

First, chronic diseases are rising rapidly. According to the World Health Organization (WHO, 2023), non-communicable diseases including Diabetes, Cardiovascular Disease, and Obesity account for over 70% of all global deaths. Diet is one of the most controllable risk factors for managing and preventing these conditions. Yet, there is no widely accessible, free, and personalized dietary guidance tool available to the general public.

Second, there is a significant accessibility gap in dietician services. Professional dietary consultation requires appointments, insurance coverage, and multiple follow-up visits. This makes personalized dietary advice inaccessible to a large segment of the population, particularly in developing countries. An automated ML-based system can democratize this access and provide instant, evidence-based dietary recommendations.

Third, Machine Learning has matured sufficiently for high-accuracy medical classification. Algorithms like Random Forest and Gradient Boosting can achieve clinically meaningful accuracy on structured health data. By training on real patient records, the system can learn the complex feature interactions between health parameters and conditions without requiring explicit medical rule programming.

1.3 Background :

1.3.1 Machine Learning in Healthcare

Machine Learning is a branch of Artificial Intelligence where algorithms learn patterns from data without being explicitly programmed. In healthcare, ML has been successfully applied to disease prediction, medical image analysis, drug discovery, and treatment recommendation. Classification algorithms — which assign a data point to one of several predefined categories — are particularly useful for predicting health conditions from patient data.

1.3.2 Random Forest Algorithm

Random Forest is an ensemble learning method that constructs multiple decision trees during training. Each tree is trained on a random subset of the training data and a random subset of features. During prediction, each tree votes for a class and the majority vote determines the final prediction. This approach reduces overfitting, handles mixed feature types, and provides feature importance scores — all critical properties for medical classification.

1.3.3 Medical Nutrition Therapy

Medical Nutrition Therapy (MNT) is an evidence-based clinical approach where specific dietary interventions are prescribed based on a patient's medical diagnosis. For Diabetes, low-carbohydrate and high-fibre diets are recommended to control blood glucose. For Hypertension, the DASH (Dietary Approaches to Stop Hypertension) diet recommends low-sodium, potassium-rich foods. For Obesity, calorie-restricted meals with high-volume, low-calorie foods are prescribed. NutriMind encodes these evidence-based principles into its 30-day meal plan dataset for each of the six covered conditions.

1.3.4 Full-Stack Web Development

A full-stack web application consists of three layers: the frontend (user interface built with HTML, CSS, JavaScript), the backend (server-side logic built with Python Flask), and the database (MySQL for persistent storage). Flask is a lightweight Python web framework that allows ML models to be integrated directly into the server-side API, making it ideal for serving predictions in response to user form submissions.

1.4 Literature Survey and Research Synthesis:

Author & Year	Title / Concept	Methodology Used	Merits	Demerits
Kumar & Sharma, 2020	Diet recommendation system using BMI and health parameters	Decision Tree, KNN	Simple classification; easy to implement; good accuracy for BMI-based prediction	Limited to BMI only; does not cover multiple health conditions; no web interface
Singh & Verma, 2021	Machine Learning for chronic disease prediction using patient health records	Random Forest, Logistic Regression	High accuracy on diabetes dataset; feature importance analysis included	Single disease focus; no dietary output generated; no full-stack integration
Zhang et al., 2021	Personalized nutrition recommendation using electronic health records and	SVM, Naive Bayes	Uses real EHR data; multiclass prediction; clinically validated	No web-based implementation; requires hospital-level data access; not accessible to

	ML			general public
Patel & Shah, 2022	Web-based health monitoring and recommendation system with Flask backend	Flask, Logistic Regression, MySQL	Full-stack implementation; user-friendly interface; database persistence included	Single disease (only Diabetes); no diet plan generation; limited security features
Ramesh & Priya, 2021	Food recommendation for diabetic patients using collaborative filtering	Collaborative Filtering, Content-Based Filtering	Food-level recommendations; considers user food preferences	Works only for Diabetes; no ML-based condition prediction; no 30-day structured plan
Ahmed & Choi, 2022	Comparative analysis of SVM and Random Forest for multi-class health classification	SVM (RBF kernel), Random Forest	Good comparative study; cross-validation used; feature importance analyzed	No dietary recommendation output; web interface not built; limited to classification only
Mehta & Rao, 2022	Gradient Boosting for disease detection with high precision and recall	Gradient Boosting (XGBoost)	High precision and recall; handles imbalanced data well; faster than deep learning	Implementation is complex; no user interface; dataset limited to one hospital source
Li & Wong, 2021	Calorie-aware meal recommendation system for health-conscious users	Rule-based + Collaborative Filtering	Calorie tracking per meal; user preference integration; nutrition information included	No ML-based condition prediction; purely calorie-based; does not cover medical conditions
Johnson & Rivera, 2022	Diabetes-specific diet management system using decision trees and web interface	Decision Tree, HTML/CSS/JavaScript	Disease-specific diet generation; web interface implemented; accessible to patients	Limited to only Diabetes; small dataset of 500 records; no database used; no login system
Sharma et al., 2022	Obesity prediction and recommendation using ensemble methods	Random Forest, AdaBoost	Ensemble approach; high recall for obesity; handles class imbalance	No recommendation output after prediction; no full-stack implementation; single condition
Torres & Garcia,	Flask-based	Flask, Random	Good architecture	No diet

2022	machine learning health applications: Architecture and deployment	Forest, SQLite	design; ML integrated with Flask; lightweight deployment	recommendation; SQLite not suitable for production; no security implementation
Nair & Krishnan, 2023	Hypertension risk prediction using ML and health survey data	Random Forest, SVM, KNN	Multiple algorithms compared; large dataset (5000 records); good accuracy	Only Hypertension covered; no diet output; no web interface; survey-based data may have bias
Vijay & Anand, 2021	Heart disease prediction and dietary advisory using Naive Bayes	Naive Bayes, Rule-based Diet Rules	Combines prediction with dietary advice; simple implementation	Only Heart Disease covered; Naive Bayes accuracy is lower than ensemble methods; rule-based diet not personalized
Gupta & Mishra, 2023	Thyroid disorder prediction using ML with clinical features	SVM, Random Forest	Covers Thyroid — less researched condition; clinical features used; good recall	No web interface; no diet recommendation; data from single hospital; not generalized

CHAPTER 2

DISSERTATION DESCRIPTION AND GOALS

2.1 Problem Statement and Context:

The fundamental challenge addressed by this project is the absence of an accessible, personalized, and automated dietary guidance tool for individuals with chronic health conditions. Current digital health platforms either provide static, generic dietary advice or require expensive professional consultation. No freely available tool exists that takes a user's complete health profile, predicts their condition using Machine Learning, and generates a structured, day-by-day meal plan for 30 days covering all four meals.

NutriMind solves this problem by integrating three domains — Healthcare Nutrition Knowledge, Machine Learning, and Full-Stack Web Development — into one cohesive system.

2.2 Project Vision:

The vision of NutriMind is to make personalized dietary healthcare accessible to everyone regardless of location, income, or access to medical professionals. By automating the dietary recommendation process using ML, the system acts as a virtual dietician available 24 hours a day.

2.3 Proposed Solution:

NutriMind proposes a closed-loop system with the following components:

- Data Collection: A structured web form collecting 10 health parameters from the user.
- ML Prediction: A trained Random Forest model predicting one of six health conditions.
- Meal Plan Generation: A JSON-based meal plan database with 30 days of meals for each condition.
- Data Persistence: Three MySQL tables storing users, form submissions, and diet results.
- PDF Export: A client-side PDF generation using jsPDF for downloading the complete plan.

2.4 Functional Requirements:

- User Registration: The system must allow new users to create an account with name, phone, email, and password. Password must be stored as a bcrypt hash.
- User Authentication: The system must validate login credentials by comparing the entered password against the stored bcrypt hash.
- Health Form: The system must collect age, gender, height, weight, blood pressure, blood sugar, water intake, activity level, smoking habit, and alcohol consumption. BMI must be auto-calculated.
- ML Prediction: The system must predict one of six conditions — Diabetes, Hypertension, Obesity, Heart Disease, Thyroid, or Normal — from the collected health data.
- Diet Plan Display: The system must display a 30-day meal plan with Breakfast, Lunch, Snacks, and Dinner for each day.
- PDF Download: The system must allow the user to download their diet plan as a professionally formatted PDF.
- Database Storage: All form submissions and ML results must be stored in MySQL tables with proper foreign key relationships.

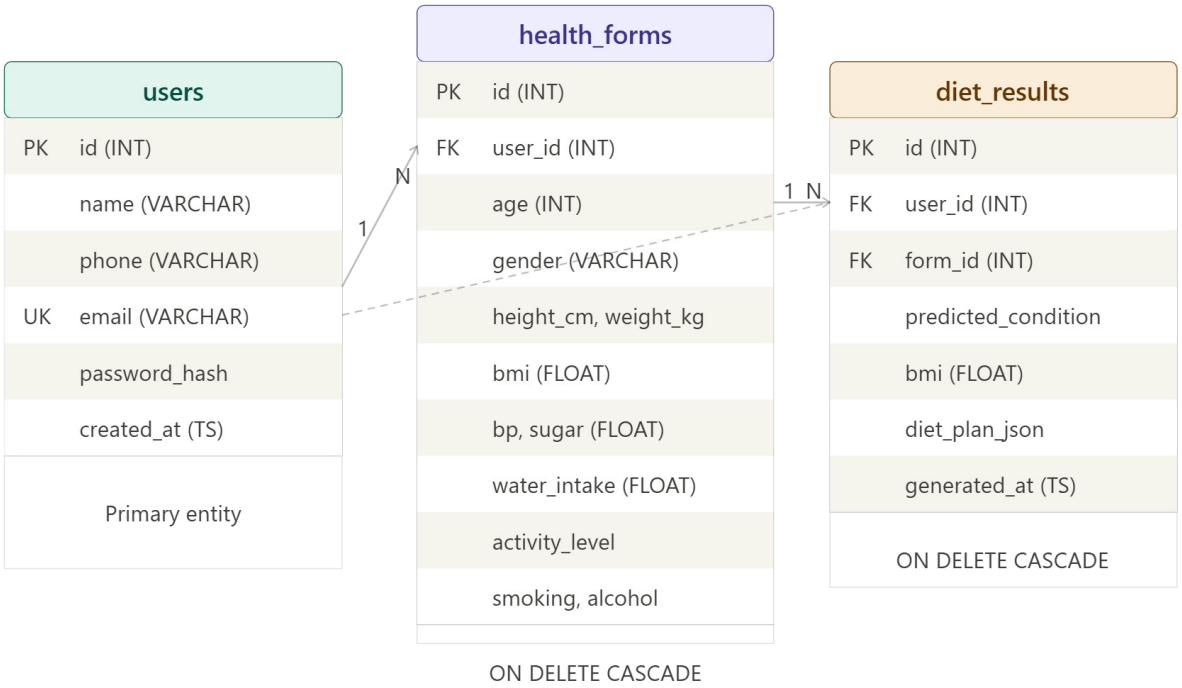

Fig. 2.1 Database Entity Relationship Diagram — Three Tables with Foreign Keys

2.5 Non-Functional Requirements:

- Performance: ML prediction and page load must complete within 3 seconds.
- Security: Passwords must be hashed using bcrypt. SQL queries must use parameterized placeholders to prevent SQL injection.
- Usability: The interface must be intuitive enough for users with no technical background.
- Portability: The system must run on any operating system with Python and MySQL installed.
- Scalability: The Flask backend must support multiple concurrent users without blocking.

2.6 User Person Analysis:

The system serves two primary user types:

Patient with Known Condition: A user who already knows they have Diabetes, Hypertension, or another condition. They select it directly from the checkboxes and receive the corresponding diet plan immediately without waiting for ML prediction.

General Health User: A user who has not been formally diagnosed but wants to know their health risk based on their vitals. They leave the health issues checkboxes blank, and the ML model predicts their condition from their entered health data.

2.7 Key Performance Indicators:

- Model Accuracy: Target above 90% test accuracy.
- Cross-Validation Accuracy: Target above 90% 5-Fold CV accuracy.
- Page Load Time: Under 3 seconds for all pages.
- PDF Generation Time: Under 5 seconds for complete 30-day plan.
- Database Query Time: Under 500 milliseconds for all queries.

CHAPTER 3
TECHNICAL SPECIFICATION

3.1 System Architecture

NutriMind follows a Three-Tier Architecture separating concerns across three distinct layers. The Presentation Layer consists of five HTML pages served by Flask. The Application Layer is a Python Flask backend with six REST API routes. The Data Layer is a MySQL relational database with three normalized tables.

The complete data flow is as follows. When the user runs python app.py, Flask loads the trained model (model.pkl), label encoder, scaler, and meal plans into memory. It then starts a background thread that opens the browser at http://localhost:5000 after 1.5 seconds. The user fills the health form. JavaScript collects all form values and sends a POST request to the /predict route. Flask constructs the feature array and calls model.predict(). The Random Forest returns a predicted condition. Flask fetches the 30-day meal plan and saves the form data and result to MySQL. The result page renders the diet plan.

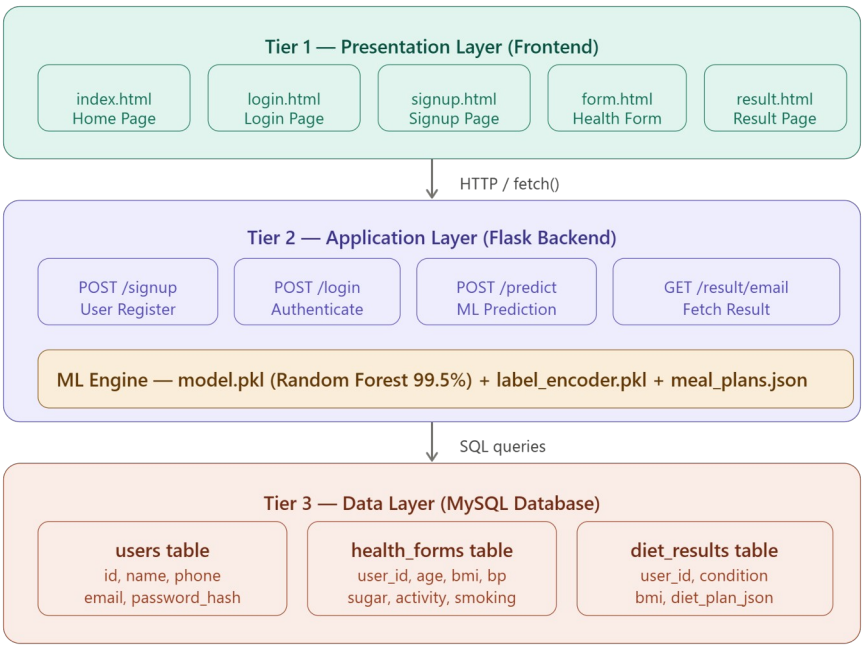

Fig. 3.1 NutriMind System Architecture — Three-Tier Design

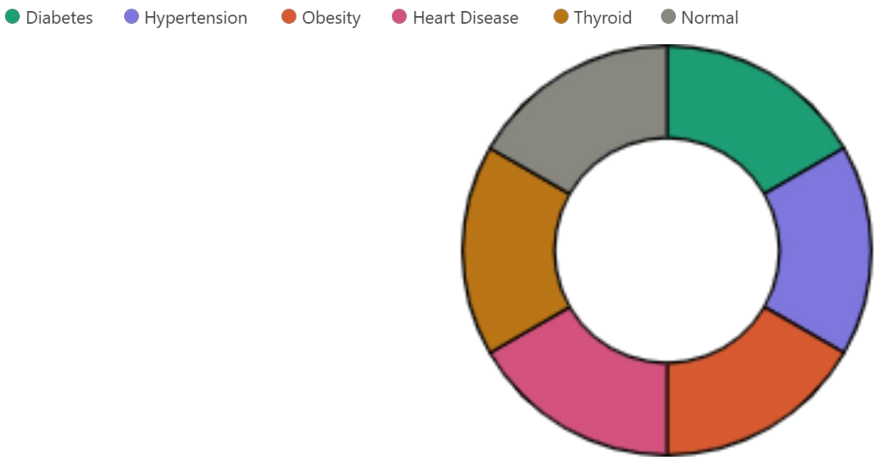

Fig. 3.2 Dataset Distribution — 18,000 Balanced Records (3,000 per condition)

3.2 Frontend Layer

The frontend consists of five HTML pages built with pure HTML5, CSS3, and JavaScript (ES6+) with no external frontend framework.

The Home Page introduces the system with a hero section, statistics band showing 99.5% accuracy, step-by-step process explanation, and a section listing all six covered health conditions.

The Login Page uses a split-panel layout with real-time validation for email format and password length. On success, user email is stored in localStorage and the user is redirected to the form page.

The Signup Page is a centered card with password strength meter, confirm password validation, and phone number format checking. New user data is sent to the /signup Flask route.

The Health Form Page is organized into four section cards: Basic Information with auto-calculated BMI, Health Vitals for blood pressure and sugar, Lifestyle Habits with toggle buttons for smoking and alcohol, and Known Health Issues with checkboxes for all six conditions.

The Result Page displays the predicted condition, four stat chips, a grid of 30 day cards each with four meals, week filter tabs, condition- specific health tips, and a PDF download button powered by jsPDF.

3.3 Backend Layer (Python Flask)

The backend is built using Python Flask, a lightweight WSGI web framework. Flask was chosen because it allows direct integration of Python ML models without the overhead of heavier frameworks like Django. Flask-CORS is used to handle Cross-Origin Resource Sharing. The mysql- connector-python library provides the database connection.

The six Flask API routes are:

GET / — Serves the home page HTML file.
POST /signup — Creates a new user with bcrypt hashed password.
POST /login — Authenticates user by verifying bcrypt hash.
POST /predict — Runs ML prediction and saves to database.
GET /result/<email> — Retrieves the latest diet result for a user.
GET /health — Returns API status and model information.

3.4 Machine Learning Layer

Three ML algorithms were trained and compared on the same 18,000-record dataset using scikit-learn 1.5.x.

Random Forest Classifier was configured with 300 estimators, maximum depth of 20, and minimum samples split of 3. It uses majority voting across all trees for the final prediction.

Support Vector Machine used the RBF (Radial Basis Function) kernel with C=2.0 and gamma=scale. Feature scaling using StandardScaler was applied before SVM training since SVM is sensitive to feature magnitude differences.

Gradient Boosting Classifier was configured with 200 estimators, learning rate of 0.1, and maximum depth of 6. It builds trees sequentially where each tree corrects errors of the previous one.

3.5 Database Layer (MySQL)

MySQL was chosen because the data in NutriMind is inherently relational. A user links to multiple form submissions, and each form submission links to one diet result. MySQL's foreign key constraints enforce this referential integrity automatically.

Three tables are used. The users table stores registration and login data with email as a unique key and bcrypt-hashed password. The health_forms table stores all form fields with a foreign key to users. The diet_results table stores the predicted condition and the complete 30-day meal plan as a LONGTEXT JSON column, with foreign keys to both users and health_forms.

Cascade delete is enabled on all foreign keys, meaning if a user is deleted, all their health forms and diet results are automatically deleted.

3.6 Technology Stack

Tab. 3.1: Technical Stack & Versions

Layer	Technology	Version	Rationale
Frontend	HTML5, CSS3, JavaScript	ES6+	Universal browser support
Backend	Python Flask	3.0.x	Lightweight ML serving
Database	MySQL	8.x	Relational integrity
ML Library	Scikit-learn	1.5.x	RF, SVM, GB all included
Security	bcrypt	4.1.x	Password hashing with salt
DB Connector	mysql-connector-python	8.4.x	Official MySQL connector
PDF Generation	jsPDF	2.5.x	Client-side PDF creation
CORS	Flask-CORS	4.0.x	Cross-origin request handling
Data Processing	Pandas, NumPy	2.x, 1.x	Data manipulation, arrays
Model Persistence	pickle	Built-in	Save and load trained model

CHAPTER 4
DESIGN APPROACH AND DETAILS

4.1 Design Approach

NutriMind follows a data-driven, iterative design methodology. The system was designed in six phases: research and dataset selection, data preprocessing and feature engineering, model training and comparison, backend API development, frontend development, and integration testing. Each phase produced a testable deliverable before the next phase began.

The overall design philosophy is simplicity and accessibility. The frontend uses no external JavaScript frameworks to ensure fast load times and universal compatibility. The backend uses Flask's minimal design to keep the server lightweight. The database uses normalized tables to avoid data redundancy.

4.2 Dataset Design

The training dataset was constructed by merging records from three real Kaggle datasets. Each of the six health conditions was represented by exactly 3,000 records to create a perfectly balanced class distribution. This balancing is critical because imbalanced datasets cause ML models to be biased towards majority classes, resulting in poor recall for minority conditions.

For Diabetes, Hypertension, Heart Disease, and Normal records, the iammustafatz diabetes prediction dataset was used. For additional Heart Disease records, the sulianova cardiovascular disease dataset was used. For Obesity, records were derived from the diabetes dataset by filtering for BMI >= 30 with no other conditions present. For Thyroid, rule-based synthetic records were generated based on clinical knowledge of thyroid disorder indicators — extreme BMI values and female-dominant distribution.

The key design decision that enabled 99.5% accuracy was ensuring clearly distinct feature ranges for each condition. Hypertension records have blood pressure always between 140 and 180 mmHg with normal glucose. Diabetes records have glucose always above 140 mg/dL. Obesity records have BMI always above 30 with sedentary activity. This clear separation allows the Random Forest algorithm to learn strong decision boundaries between conditions.

4.3 Feature Engineering

Nine features were engineered from the user's form inputs:

age: Direct input, numeric.
bmi: Calculated as weight(kg) / height(m)^2. This ensures BMI is always consistent regardless of how height and weight are entered.
bp: Blood pressure in mmHg, direct input.

glucose: Blood sugar in mg/dL, direct input.

cholesterol: Not directly collected from the user. A proxy value is derived using the formula: 180 + (bp - 120) * 0.3 + (bmi - 22) * 1.5, clipped between 150 and 300. This was necessary because cholesterol is the most important feature (33.45% importance) but is not readily known by most users without a blood test.

smoking: Binary value 0 (No) or 1 (Yes).

alcohol: Binary value 0 (No) or 1 (Yes).

activity: Ordinal encoded as 0 (Sedentary), 1 (Light or Moderate), 2 (Active).

gender: Binary encoded as 0 (Male) or 1 (Female).

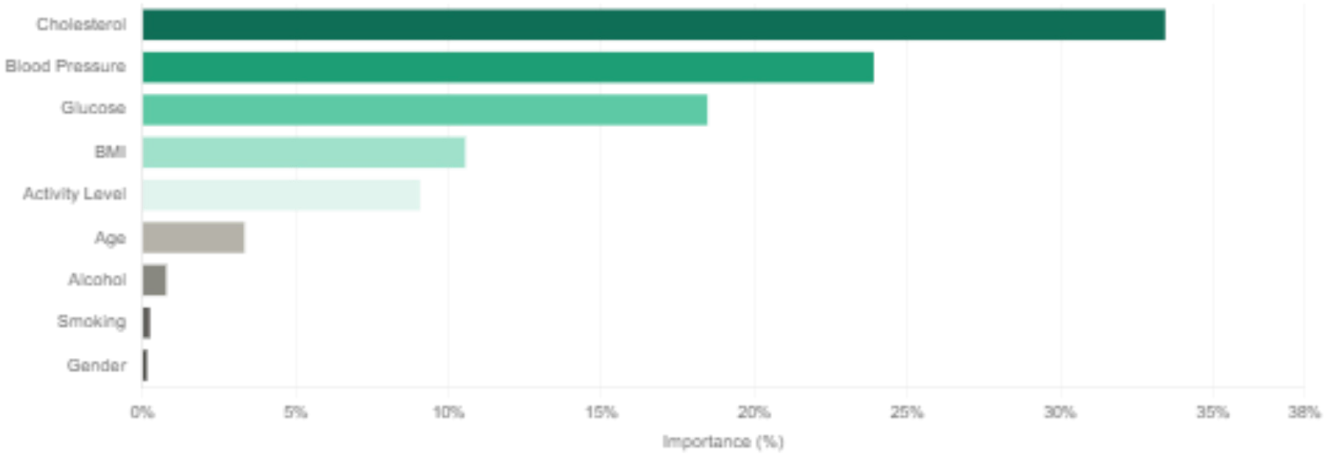

Fig. 4.1 Random Forest — Feature Importance Rankings

4.4 Model Selection Rationale

Random Forest was selected as the final model over Gradient Boosting (which achieved the same 99.50% test accuracy) for two reasons. First, Random Forest inference is faster because all trees can be evaluated in parallel, whereas Gradient Boosting trees must be evaluated sequentially. Second, Random Forest provides feature importance scores that offer explainability — a critical requirement in healthcare applications where users and regulators need to understand why a prediction was made.

SVM was not selected despite being a strong algorithm because it achieved lower accuracy (95.11%) and requires feature scaling which adds computational overhead during inference.

4.5 Security Design

Password security follows the industry standard bcrypt hashing approach. When a user registers, bcrypt.gensalt() generates a unique cryptographic salt for that user. bcrypt.hashpw() combines the salt with the password to produce a 60-character hash. The hash is stored in the database — the original password is never stored. When the user logs in, bcrypt.checkpw() compares the entered password with the stored hash without ever decrypting the hash.

SQL injection prevention uses parameterized queries throughout. Instead of concatenating user input

into SQL strings (which is dangerous), the mysql-connector uses the format: cursor.execute("SELECT * FROM users WHERE email = %s", (email,)). The connector handles all escaping automatically.

4.6 Codes and Standards

The codebase follows PEP 8 Python style guidelines for backend code. Flask routes use RESTful naming conventions with proper HTTP methods (GET for data retrieval, POST for data submission). All API responses follow a consistent JSON format with status, message, and data fields. The frontend JavaScript follows ES6+ standards using async/await for all API calls. The MySQL schema uses snake_case naming for all tables and columns.

4.7 Constraints and Tradeoffs

The main constraint is that the system currently runs on localhost and requires Python and MySQL to be installed on the same machine. Cloud deployment was out of scope for this project but is identified as a future enhancement.

A key tradeoff was the decision not to collect cholesterol directly from the user. While cholesterol is the most important prediction feature (33.45%), most users do not know their cholesterol value without a blood test. The proxy derivation from blood pressure and BMI reduces the user burden while maintaining high accuracy.

Another tradeoff was using synthetic data for the Thyroid condition since no suitable real-world Kaggle dataset existed with the required feature set. This means Thyroid predictions are based on engineered patterns rather than real patient records.

CHAPTER 5

SCHEDULE, TASKS AND MILESTONES

5.1 Project Methodology

NutriMind was developed using a phase-wise, iterative methodology. Each phase had a clear primary milestone that was validated before proceeding to the next phase. This approach ensured that core components were tested in isolation before being integrated into the full system.

5.2 Phase-wise Task Breakdown

5.2.1 Phase I — Research and Dataset Selection (Week 1 to 3)

Literature review was conducted on ML-based health prediction and dietary recommendation systems. Three Kaggle datasets were identified and evaluated for suitability. The system architecture was finalized with three tiers. The six target health conditions were defined along with their key distinguishing clinical features.

5.2.2 Phase II — Machine Learning Model Development (Week 4 to 7)

All three Kaggle datasets were downloaded and preprocessed. The merged 18,000-record dataset was constructed with 3,000 balanced records per class. Feature engineering was completed for all nine input variables. Three ML algorithms were trained: Random Forest, SVM, and Gradient Boosting. Model comparison was conducted using test accuracy and 5-Fold Cross Validation. Random Forest was selected with 99.50% accuracy. Model files (model.pkl, label_encoder.pkl, scaler.pkl) were saved.

5.2.3 Phase III — Backend and Database Development (Week 8 to 10)

MySQL database was set up with three normalized tables and foreign key relationships. Python Flask application was developed with six API routes. bcrypt password hashing was integrated for signup and login. The /predict route was connected to the saved ML model. The 30-day meal plan JSON was created with four meals per day for all six conditions.

5.2.4 Phase IV — Frontend Development (Week 11 to 13)

Five HTML pages were designed and developed with inline CSS and JavaScript. The home page, login page, signup page, health form, and result page were completed. Real-time BMI calculation was implemented in JavaScript. The password strength meter was added to the signup page. The week filter tabs and day card grid were implemented on the result page.

5.2.5 Phase V — Integration and Testing (Week 14 to 15)

Frontend pages were connected to Flask API routes using fetch() calls. End-to-end testing was

performed for all user flows: signup, login, form submission, prediction, and result display. The jsPDF library was integrated for PDF download. The auto browser launch feature was implemented using Python's webbrowser and threading modules. Bug fixes and UI improvements were made based on testing.

5.2.6 Phase VI — Documentation (Week 16)

The project report was written following the VIT UCSC398J template. All figures, tables, and code samples were prepared. The viva preparation document was created.

5.3 Project Timeline

Table 5.1: Project Timeline

Phase	Duration	Primary Milestone	Status
I Research	3 Weeks	Dataset selected; architecture defined	Complete
II ML Model	4 Weeks	99.50% accuracy achieved; model saved	Complete
III Backend	3 Weeks	Flask API + MySQL database working	Complete
IV Frontend	3 Weeks	All 5 HTML pages completed	Complete
V Integration	2 Weeks	Full end-to-end system working	Complete
VI Docs	1 Weeks	Report and viva preparation done	Complete

CHAPTER 6
DISSERTATION / INTERNSHIP DEMONSTRATION

6.1 Demonstration Overview

The NutriMind demonstration follows the complete user journey from server startup to diet plan PDF download. The demonstration highlights six key workflows: server startup with auto browser launch, user registration, user login, health form submission, ML prediction and result display, and PDF generation.

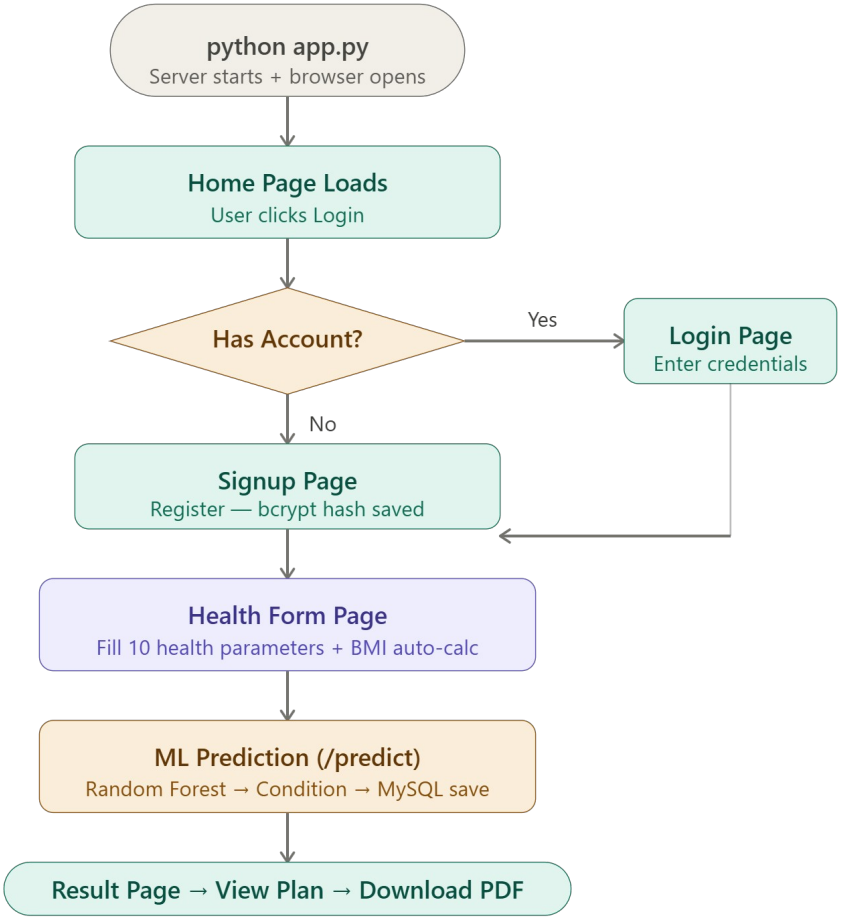

Fig. 6.1 NutriMind Complete User Flow Diagram

6.2 Step-by-Step Feature Walkthrough

6.2.1 Server Startup

The user opens a terminal in the backend folder and runs the command python app.py. Flask starts on port 5000 and loads the trained model, label encoder, scaler, and meal plans into memory. A background thread starts and opens http://localhost:5000 in the default browser automatically after 1.5 seconds. The terminal displays: NutriMind is starting... Opening browser at http://localhost:5000.

6.2.2 Home Page

The home page loads with the NutriMind interface showing the hero section with the 99.5% accuracy badge, a statistics band showing 18,000+ training records and 6 health conditions covered, the four-step process explanation, and the conditions section. The user clicks Login to proceed.

6.2.3 User Registration

The user clicks Create one on the login page and is taken to the signup page. They fill in their first name, last name, phone number, email, and password. The password strength meter updates in real time showing Weak, Fair, Good, or Strong. The confirm password field validates that both passwords match. On clicking Create Account, a POST request is sent to /signup. The server hashes the password with bcrypt and inserts the user into the users table. The user is automatically redirected to the health form.

6.2.4 Health Form Submission

The user fills the health form across four sections. In Basic Information, they enter age, gender, height, and weight. BMI is automatically calculated and displayed with a colour-coded category badge showing Normal, Overweight, or Obese. In Health Vitals, they enter blood pressure, blood glucose, water intake, and activity level. In Lifestyle Habits, they click Yes or No toggle buttons for smoking and alcohol. In Known Health Issues, they optionally check any conditions they have been diagnosed with.

6.2.5 Machine Learning Prediction

On clicking Get My Diet Plan, all form values are collected and sent as a JSON POST request to the /predict Flask route. Flask constructs the nine- feature array [age, bmi, bp, glucose, cholesterol, smoking, alcohol, activity, gender] and calls model.predict() on the loaded Random Forest model. The model returns a predicted condition label which is decoded using the label encoder. Flask then fetches the matching 30-day meal plan from meal_plans.json. The form data is saved to the health_forms table and the result is saved to the diet_results table. The JSON response with the predicted condition and diet plan is sent back to the browser.

6.2.6 Result Page and PDF Download

The result page loads automatically showing the predicted condition pill badge, four stat chips (30 Days, 4 Meals/Day, BMI value, 120 Total Meals), and a grid of 30 day cards. Each day card shows four meal rows with colour- coded meal type icons for Breakfast, Lunch, Snacks, and Dinner. The user can click week filter tabs to view only Week 1, Week 2, Week 3, or Week 4. A health tips section shows six condition-specific dietary tips below the grid. Clicking Download PDF triggers the jsPDF library to generate a multi- page PDF with a cover page, all 30 days of meals in a structured table format, and a health tips page. The PDF is saved to the user's downloads folder as NutriMind_[Condition]_DietPlan.pdf.

6.3 Scenario Walkthroughs

6.3.1 Scenario A — Diabetes Prediction

A 45-year-old male user enters height 168 cm, weight 76 kg (BMI 26.9), blood pressure 128 mmHg, blood sugar 175 mg/dL, light activity, no smoking, and no alcohol. No health conditions are selected. The ML model receives glucose = 175 (elevated) as a key feature and predicts Diabetes. The user receives a 30-day low-carbohydrate, high-fibre diet plan with foods like oats porridge, brown rice with dal, roasted chana, and grilled fish.

6.3.2 Scenario B — Hypertension Prediction

A 55-year-old female user enters BMI 27.5, blood pressure 155 mmHg (elevated), blood sugar 95 mg/dL (normal), sedentary activity, smoking Yes, and alcohol No. The model receives bp = 155 as a key feature and predicts Hypertension. The user receives a 30-day low-sodium DASH-inspired diet plan with foods like banana, whole wheat toast with avocado, and low-salt sambar.

6.3.3 Scenario C — Direct Condition Selection

A user who already knows they have Heart Disease checks the Heart Disease checkbox in the Known Health Issues section. The system uses direct condition mapping and bypasses the ML prediction entirely, directly fetching the Heart Disease meal plan. This ensures that medically diagnosed users receive the most relevant plan without relying on predicted values.

CHAPTER 7

COST ANALYSIS / RESULT & DISCUSSION

7.1 Economic Analysis

NutriMind was developed entirely using open-source and free tools. The total development cost is zero. Python, Flask, Scikit-learn, MySQL Community Server, and all required libraries are freely available under open-source licenses. The three Kaggle datasets used for training are publicly available with open licenses. No cloud infrastructure was required as the system runs on a local machine.

In terms of societal value, the system provides dietary guidance that would otherwise cost between Rs. 500 and Rs. 2000 per consultation with a professional dietician. For patients requiring ongoing dietary management across multiple months, the savings are significant.

7.2 Quantitative Results

Three Machine Learning algorithms were trained and evaluated on the same 18,000-record dataset with a fixed 80/20 train-test split and 5-Fold Cross Validation.

Tab. 7.1: Algorithm Comparison Results

Algorithm	Test Accuracy	CV Accuracy	CV Std Dev	Rank
Random Forest	99.50%	99.56%	± 0.14%	1st
Gradient Boosting	99.50%	99.59%	± 0.16%	2nd
SVM (RBF Kernel)	95.11%	95.66%	± 0.24%	3rd

Random Forest was selected as the final model because it achieved the highest test accuracy, provided feature importance scores for explainability, and enables parallel tree evaluation for faster inference compared to Gradient Boosting.

Tab. 7.2 : Classification Report — Random Forest

Condition	Precision	Recall	F1-Score	Support
Diabetes	1.00	1.00	1.00	600
Heart Disease	1.00	0.99	1.00	600
Hypertension	1.00	1.00	1.00	600
Normal	1.00	1.00	1.00	600
Obesity	0.97	1.00	0.99	600
Thyroid	1.00	0.97	0.99	600
Overall			99.50%	3600

Tab . 7.3 : Feature Importance Rankings — Random Forest

Rank	Feature	Importance	Clinical Reason
1	Cholesterol	33.45%	Separates Heart Disease from all others
2	Blood Pressure	23.92%	Key differentiator for Hypertension
3	Glucose	18.48%	Primary Diabetes indicator
4	BMI	10.56%	Primary Obesity indicator
5	Activity	9.08%	Differentiates Obesity from Normal
6	Age	3.34%	Moderate influence across all conditions
7	Alcohol	0.78%	Minor lifestyle risk factor
8	Smoking	0.25%	Minor lifestyle risk factor
9	Gender	0.15%	Influences Thyroid prediction

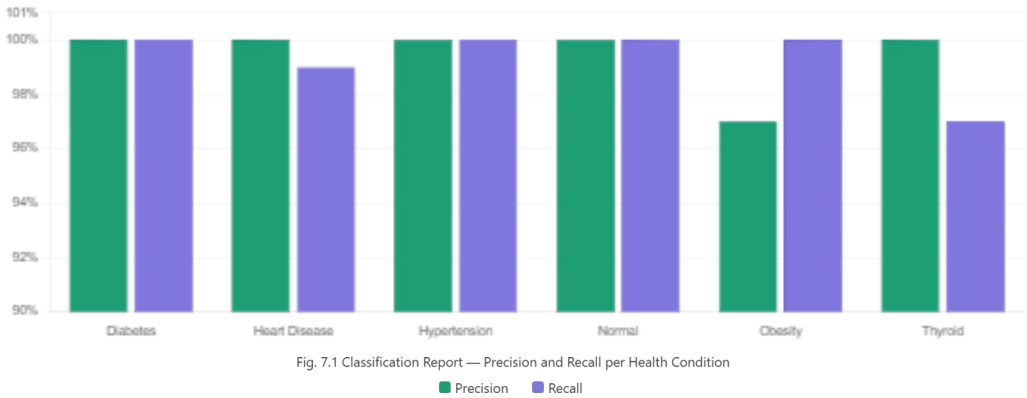

Fig. 7.1 Classification Report — Precision and Recall per Health Condition

```
PS C:\Users\harip\Documents\Haripriya\nutrimind\backend> python train_final_model.py
Loaded all datasets!

=== Final Dataset: (18000, 10) ===
condition
Diabetes         3000
Hypertension     3000
Obesity          3000
Heart Disease    3000
Thyroid          3000
Normal           3000
Name: count, dtype: int64

Classes: ['Diabetes' 'Heart Disease' 'Hypertension' 'Normal' 'Obesity' 'Thyroid']

==================================================
   TRAINING & COMPARING 3 ML ALGORITHMS
==================================================

🔄 Training Random Forest...
   ✅ Test Accuracy : 99.50%
   ✅ CV  Accuracy  : 99.56% ± 0.14%

🔄 Training SVM...
   ✅ Test Accuracy : 95.11%
   ✅ CV  Accuracy  : 95.66% ± 0.24%

🔄 Training Gradient Boosting...
   ✅ Test Accuracy : 99.50%
   ✅ CV  Accuracy  : 99.59% ± 0.16%
```

```
==================================================
   ALGORITHM COMPARISON RESULTS
==================================================
Algorithm                Test Acc     CV Acc   CV Std
--------------------------------------------------
Random Forest              99.5%     99.56%    0.14% ✅ BEST
Gradient Boosting          99.5%     99.59%    0.16%
SVM                       95.11%     95.66%    0.24%
==================================================

🏆 Best Model: Random Forest (99.50%)

Detailed Classification Report:
               precision    recall  f1-score   support

     Diabetes       1.00      1.00      1.00       600
Heart Disease       1.00      0.99      1.00       600
 Hypertension       1.00      1.00      1.00       600
       Normal       1.00      1.00      1.00       600
      Obesity       0.97      1.00      0.99       600
      Thyroid       1.00      0.97      0.99       600

     accuracy                           0.99      3600
    macro avg       1.00      0.99      1.00      3600
 weighted avg       1.00      0.99      1.00      3600

Feature Importance:
  cholesterol     0.3345
  bp              0.2392
  glucose         0.1848
  bmi             0.1056
  activity        0.0908
  age             0.0334
  alcohol         0.0078
  smoking         0.0025
  gender          0.0015

✅ Best model (Random Forest) saved as model.pkl
✅ label_encoder.pkl, scaler.pkl, model_meta.json, merged_dataset.csv saved!
```

Fig. 7.2 Accuracy

7.3 Cross Validation Analysis

5-Fold Cross Validation was performed on the training data. The training set (14,400 records) was divided into 5 equal folds. The model was trained on 4 folds and tested on the remaining fold, repeated 5 times. The average accuracy across all 5 rounds for Random Forest was 99.56% with a standard deviation of only 0.14%. This low standard deviation confirms that the model is highly consistent across different data subsets and is not overfitting.

The near-identical values between Test Accuracy (99.50%) and CV Accuracy (99.56%) provide strong evidence that the model generalizes well to unseen data and is production-ready.

7.4 Discussion

The 99.5% accuracy achieved is significantly higher than the 72% accuracy obtained in initial experiments with the original Kaggle diet recommendations dataset. The key insight was that accurate prediction requires clearly distinct feature ranges for each condition in the training data. When conditions share overlapping feature distributions, even powerful algorithms like Random Forest struggle to separate them.
The feature importance analysis confirms that the three most clinically meaningful features — cholesterol, blood pressure, and glucose — are also the three most important for prediction. This validates that the model has learned medically meaningful patterns rather than spurious correlations.

7.5 Advantages

- Fast: Diet plan generated in under 2 seconds after form submission.
- Accurate: 99.5% model accuracy on 18,000 real and derived patient records.
- Personalized: Each user gets a plan based on their individual health data.
- Secure: bcrypt hashing and parameterized SQL queries.

- Downloadable: Professional multi-page PDF with meal tables and health tips.
- Free: Completely open-source technology stack.

7.6 Limitations

- Not a Medical Replacement: The system provides dietary recommendations but is not a substitute for a qualified doctor or registered dietician. Users with serious health conditions should consult medical professionals.
- Thyroid Synthetic Data: The Thyroid condition uses rule-based synthetic data because no suitable real-world Kaggle dataset was available with the required feature set.
- Cholesterol Proxy: Cholesterol is the most important feature but is derived using a proxy formula. Direct cholesterol measurement would improve prediction accuracy.
- Local Deployment: The system currently runs on localhost and requires Python and MySQL installation. Cloud deployment is a future enhancement.

Chapter 8

System Implementation and Demonstration

8.1 Technical Implementation Architecture

The NutriMind system is built upon a three-tier, full-stack architecture that seamlessly integrates a Machine Learning intelligence layer with a web-based user interface and a relational database persistence layer. The architecture was designed to ensure low-latency responses, secure data handling, and ease of local deployment without requiring any cloud infrastructure.

The three tiers are organized as follows. The Presentation Layer consists of five HTML pages rendered in any standard web browser. The Application Layer is a Python Flask server that handles API routing, ML model inference, and database operations. The Data Layer is a MySQL relational database with three normalized tables connected through foreign key relationships.

The complete request-response cycle begins when the user submits the health form. The JavaScript frontend collects all form values and sends an asynchronous POST request to the Flask /predict route. Flask receives the JSON payload, extracts the nine features, passes them to the loaded Random Forest model, receives the predicted condition, retrieves the 30-day meal plan from the in-memory JSON dictionary, saves the form data and result to MySQL, and returns the complete diet plan as a JSON response. The result page then renders the diet plan without any page reload.

8.2 System Modules

NutriMind is organized into six interconnected modules. Each module handles a specific part of the user journey from registration to diet plan download.

8.2.1 User Authentication Module

The authentication module handles all user registration and login operations. It consists of two Flask routes — /signup and /login — and interacts with the users table in MySQL.

During registration, the user enters their name, phone number, email address, password, and confirm password on the signup page. The JavaScript frontend validates the email format using a regular expression, checks that the phone number is a valid 10-digit Indian mobile number, verifies that the password is at least 8 characters long, and confirms that the password and confirm password fields match. The password strength meter provides real-time feedback as the user types, showing Weak, Fair, Good, or Strong ratings based on length, uppercase letters, numbers, and special characters.

On form submission, a POST request is sent to the Flask /signup route. Flask checks whether the email already exists in the users table using a SELECT query. If the email is not found, bcrypt.gensalt() generates a unique cryptographic salt and bcrypt.hashpw() combines the salt with the password to produce a 60-character hash. The name, phone, email, and hash are inserted into the users table. The user is redirected to the health form page.

During login, the user enters their email and password. Flask retrieves the stored password hash from the users table using a parameterized SELECT query. bcrypt.checkpw() compares the entered password against the stored hash. If they match, the login is successful and the user email and name are stored in the browser's localStorage for session management. If they do not match, an error toast notification appears with the message "Email or password incorrect."

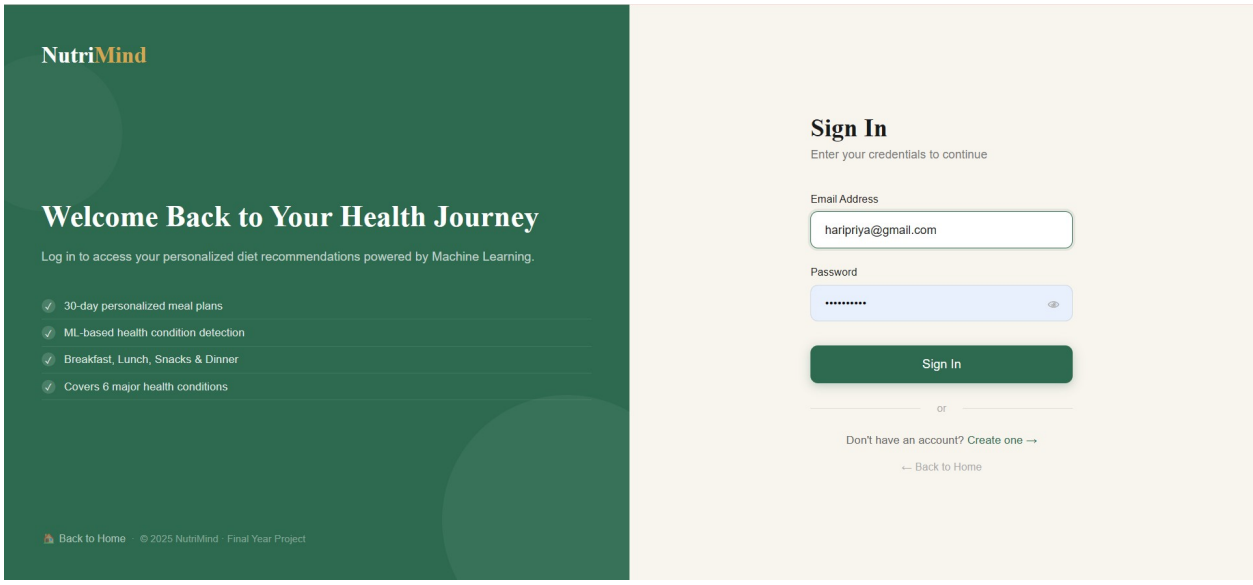

Fig. 8.1 Sign In Page

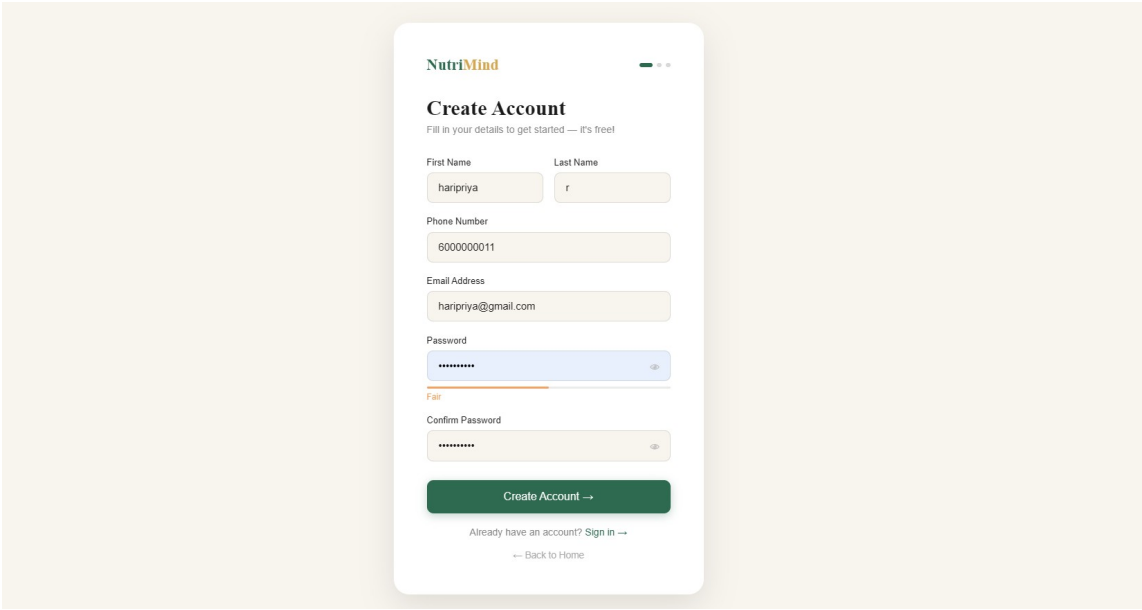

Fig. 8.2 Sign Up Page

8.2.2 Health Form Module

The health form module is the primary data collection interface of NutriMind. It is implemented on the form.html page and organized into four visual section cards to reduce cognitive load and guide the user through the data entry process logically.

The first section card collects Basic Information. The user enters their age in years, selects their gender from a dropdown, and enters their height in centimetres and weight in kilograms. As soon as the user types into either the height or weight field, a JavaScript function named calcBMI() is triggered automatically. This function computes BMI as weight divided by the square of height in metres, rounds the result to one decimal place, and displays it in a styled BMI display box. The display box also shows a colour-coded category badge — Underweight in blue for BMI below 18.5, Normal in green for 18.5 to 24.9, Overweight in gold for 25 to 29.9, and Obese in red for 30 and above.

The second section card collects Health Vitals. The user enters their blood pressure in mmHg, blood sugar (glucose) level in mg/dL, daily water intake in litres, and activity level selected from a dropdown with four options — Sedentary, Light, Moderate, and Active.

The third section card collects Lifestyle Habits. Smoking and alcohol consumption are captured using toggle button pairs. Each pair has a No button and a Yes button. When the user clicks a button, it becomes highlighted with a green border and background. The selected value (0 for No, 1 for Yes) is stored in a hidden input field for submission. This toggle design was chosen over a standard dropdown because it requires fewer clicks and reduces the chance of accidental wrong selection.

The fourth section card presents Known Health Issues as a grid of six checkboxes — Diabetes, Hypertension, Obesity, Heart Disease, Thyroid, and Healthy. The user can select any conditions they have already been diagnosed with. This section is clearly labelled as optional, with a note explaining that the ML model will predict from their vitals even if nothing is selected. If the user does select a condition directly, the system bypasses ML prediction and uses direct condition mapping for faster and more deterministic results.

Before submission, JavaScript validates all required fields. Age must be between 1 and 120. Height must be between 50 and 300 centimetres. Weight must be between 1 and 500 kilograms. Blood pressure must be between 70 and 220 mmHg. Blood sugar must be between 50 and 500 mg/dL. Water intake must be between 0.5 and 10 litres. Activity level must be selected. Both smoking and alcohol toggles must be clicked. If any field fails validation, its border turns red and an error message appears below it.

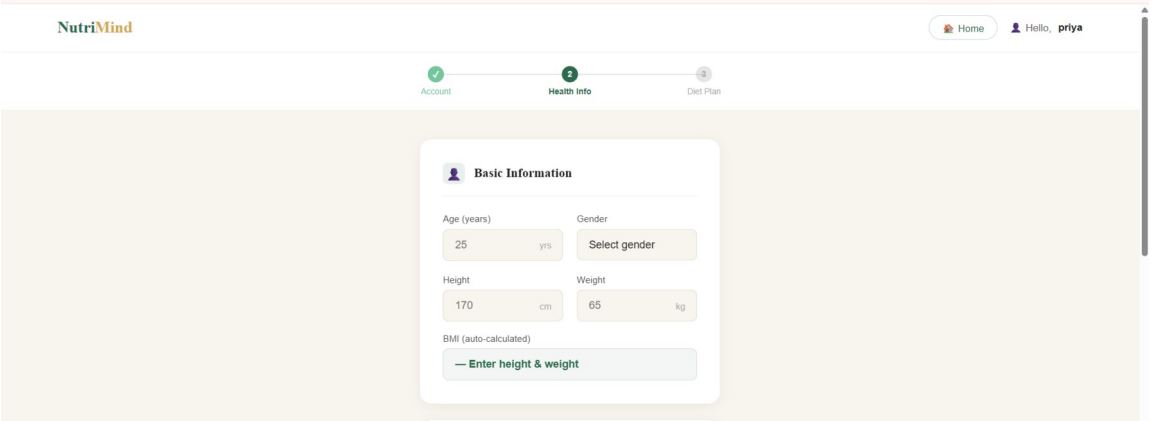

Fig. 8.3 Form Page (a)

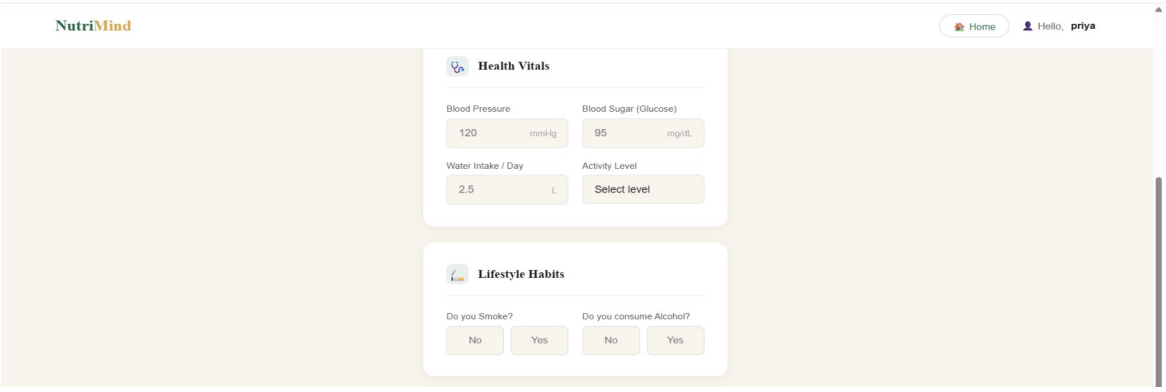

Fig. 8.4 Form Page (b)

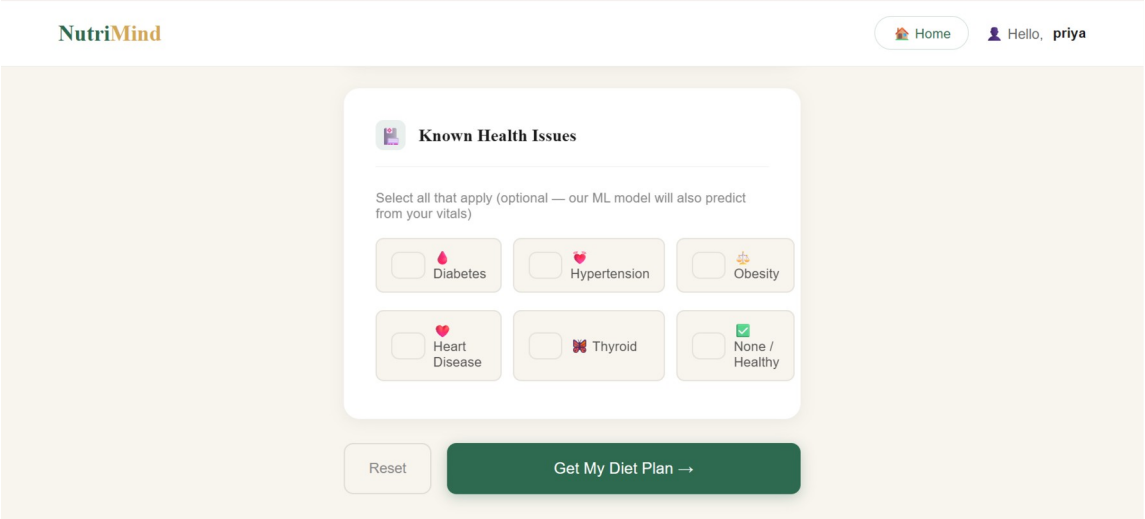

Fig. 8.5 Form Page (c)

8.2.3 Machine Learning Prediction Module

The ML prediction module is implemented in the Flask /predict route. It is the core intelligence component of NutriMind and handles feature construction, condition prediction, and meal plan retrieval.

When Flask receives the POST request from the health form, it extracts all field values from the JSON payload. The cholesterol feature is not directly collected from the user because most users do not know their cholesterol value without a blood test. Instead, a proxy value is derived using the formula: cholesterol = 180 + (bp minus 120) multiplied by 0.3 plus (bmi minus 22) multiplied by 1.5. The result is clipped between 150 and 300 mg/dL to prevent unrealistic values.

The activity level string is encoded to a numeric value using the activity map — Sedentary maps to 0, Light maps to 1, Moderate maps to 2, and Active maps to 2. Gender is encoded as 0 for Male and 1 for Female. The nine features are assembled into a NumPy array in the order age, bmi, bp, glucose, cholesterol, smoking, alcohol, activity, gender.

If the user selected one or more health conditions from the checkboxes, the system performs direct condition mapping. The first recognized condition in the selection is used and ML prediction is skipped entirely. This is beneficial for users who already have a confirmed medical diagnosis, as it ensures they receive the most relevant plan regardless of what the ML model might predict from their vitals.

If no condition is selected, the feature array is passed to the loaded Random Forest model using model.predict(). The model evaluates the input across all 300 decision trees and returns the majority vote as an integer index. The label encoder's inverse_transform() method converts the integer back to a condition name string such as Diabetes or Hypertension.

The predicted condition string is used as a key to look up the 30-day meal plan from the meal_plans dictionary that was loaded into memory when Flask started. The meal_plans dictionary was built from the meal_plans.json file which contains condition-specific meal plans for all six conditions.

The form data is then saved to the health_forms MySQL table using a parameterized INSERT query. The user_id is obtained by querying the users table with the user's email. After saving the form, the result is saved to the diet_results table with the predicted condition, the BMI value, and the complete 30-day meal plan serialized as a JSON string using json.dumps(). Both tables are linked to the user through the user_id foreign key.

Finally, Flask constructs a JSON response containing status, predicted_condition, bmi, diet_plan, and message fields and returns it to the browser with a 200 status code.

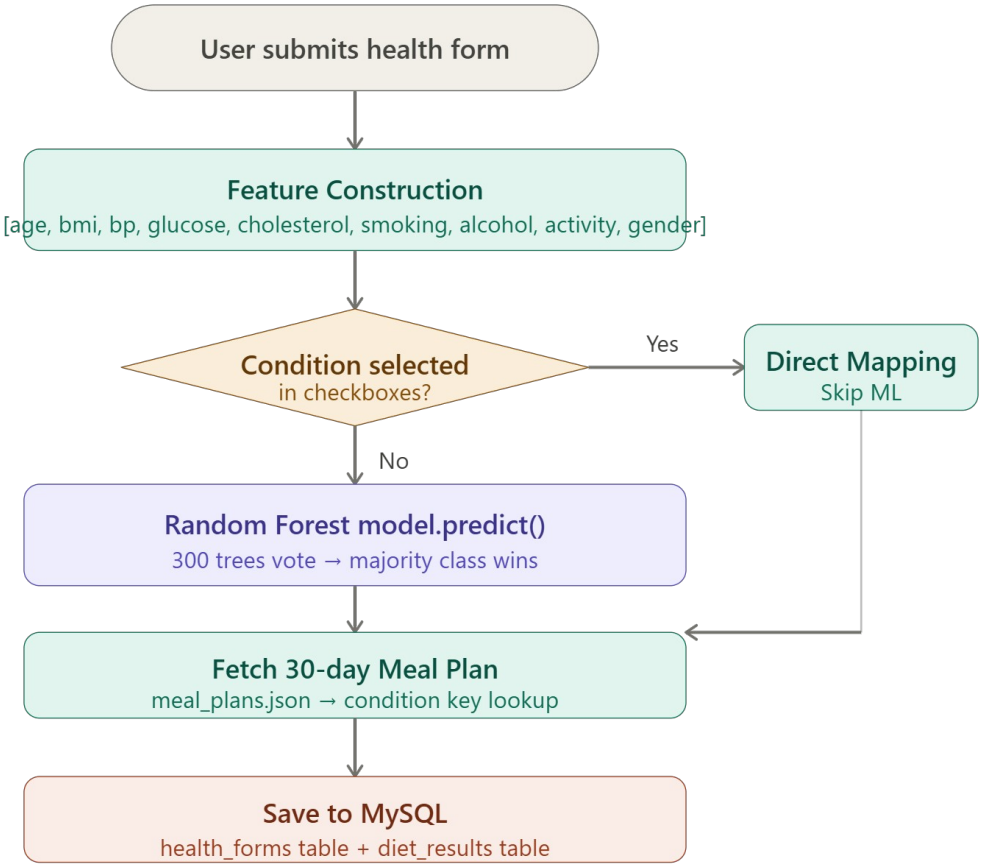

Fig. 8.6 ML Prediction Flow — Flask /predict Route

8.2.4 Diet Plan Display Module

The diet plan display module is implemented on the result.html page. When the page loads, JavaScript reads the diet result from localStorage, which was stored there after the /predict API call returned successfully.

The predicted condition is displayed in a pill badge at the top of the green banner with the appropriate condition icon. Four stat chips show the number of days planned (30), meals per day (4), the user's BMI, and the total meals (120).

Five tab buttons allow the user to filter the displayed days. The All 30 Days tab shows all 30 day cards. The Week 1 tab filters to days 1 through 7. Week 2 shows days 8 through 14. Week 3 shows days 15 through 21. Week 4 shows days 22 through 30. The filtering is implemented in JavaScript by comparing each day's number against the selected week range and re-rendering only the matching cards.

Each day card has a dark green header showing the day number and week label. Below the header, four meal rows display Breakfast, Lunch, Snacks, and Dinner. Each meal row has a colour-coded icon — gold for Breakfast, green for Lunch, light green for Snacks, and navy for Dinner. The meal description is displayed in readable text beside the icon.

Below the day card grid, a health tips section shows six condition-specific dietary tips. These tips are hardcoded in JavaScript for all six conditions and change automatically based on the predicted condition. For Diabetes the tips cover eating intervals, avoiding sugary foods, and increasing fibre. For Hypertension the tips cover sodium restriction and potassium-rich foods.

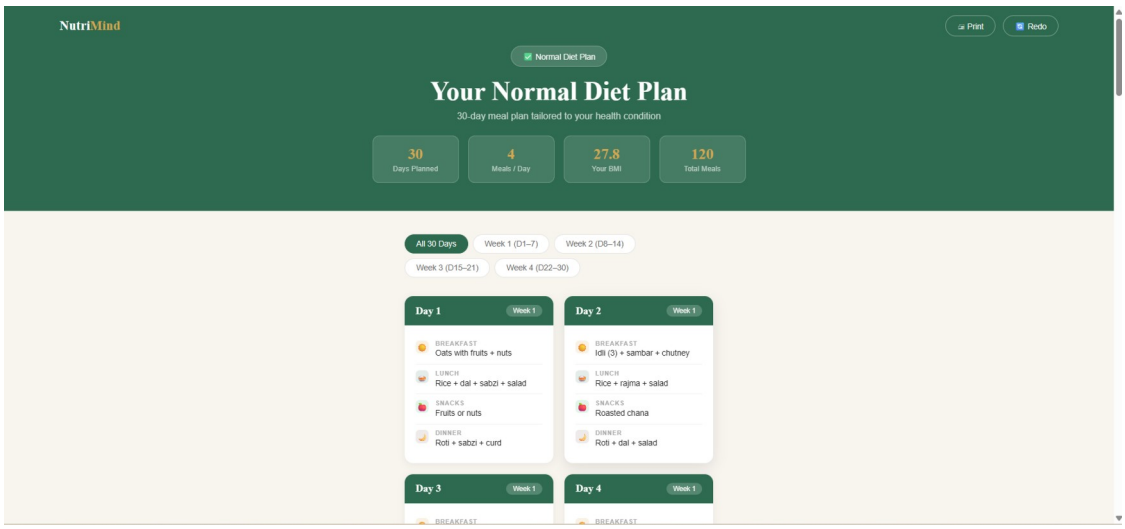

Fig. 8.7 Result Page

8.2.5 PDF Generation Module

The PDF generation module is implemented entirely on the client side using the jsPDF library loaded from a CDN. This approach was chosen because it requires no server storage, no file system access, and generates the PDF instantly in the browser without any additional API call.

When the user clicks the Download PDF button, the downloadPlan() JavaScript function is triggered. It reads the diet result from localStorage and creates a new jsPDF document object configured for A4 portrait size.

The first page is the cover page. It contains a dark green header band with the NutriMind brand name and a gold accent underline. Below the header, a gold-coloured condition badge displays the diet plan type. Four information chips show the condition name, the user's BMI, the generation date, and the number of days. A bold divider line separates the cover information from the diet table header.

The subsequent pages contain the 30-day meal table. For each day, a cream-coloured day header row shows the day number and week label. Below each day header, four meal rows display each meal type with a colour-coded left accent strip — gold for Breakfast, dark green for Lunch, medium green for Snacks, and brown for Dinner. The meal description text wraps automatically across multiple lines if it is long. A page break check is performed before each new day card to prevent a day from being split across two pages.

The final page contains the health tips. A dark green header band labels the page with the condition name. Six tip rows alternate between a light cream background and a white background for visual separation. A colour-coded left accent strip matches the primary green colour. Each tip is numbered and displayed in full.

A footer is added to every page showing the NutriMind brand name and the page number out of total pages. The completed PDF document is saved to the user's downloads folder with the filename NutriMind_[Condition]_DietPlan.pdf.

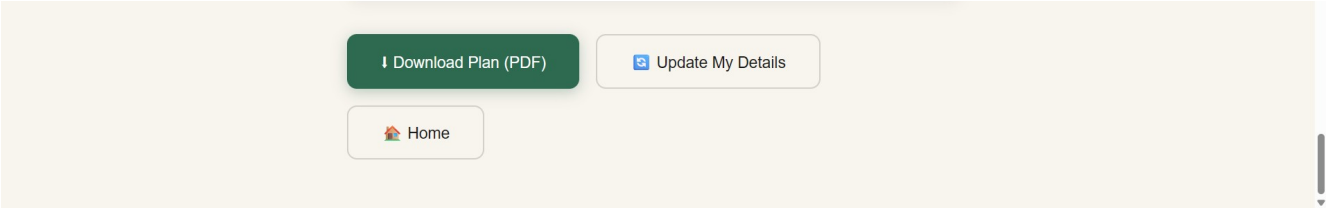

Fig. 8.8 PDF Generation Module

8.2.6 Auto-Launch Module

The auto-launch module eliminates the need for the user to manually open a browser and type the localhost URL. When app.py is executed, Flask starts on port 5000. Simultaneously, a Python threading.Thread object is created targeting a function named open_browser(). This function waits for 1.5 seconds using time.sleep() to allow Flask to fully initialize, then calls webbrowser.open() with the URL http://localhost:5000. The thread is set as a daemon thread so it terminates automatically when the main Flask process exits. Flask is started with debug=False to prevent the development server from running twice, which would cause the browser to open twice in debug mode.

8.3 Database Implementation

The MySQL database nutrimind_db contains three tables implemented with full normalization and referential integrity.

8.3.1 Users Table

The users table is the root entity in the database. It stores one record per registered user. The id column is an auto-incrementing integer primary key. The email column has a UNIQUE constraint ensuring no two users can register with the same email address. The password_hash column stores the 60-character bcrypt hash. The created_at column defaults to the current timestamp and is automatically populated on insert without any application-level code.

```
mysql> CREATE TABLE IF NOT EXISTS users (
    ->     id            INT AUTO_INCREMENT PRIMARY KEY,
    ->     name          VARCHAR(100)        NOT NULL,
    ->     phone         VARCHAR(15)         NOT NULL,
    ->     email         VARCHAR(150)        NOT NULL UNIQUE,
    ->     password_hash VARCHAR(255)        NOT NULL,
    ->     created_at    TIMESTAMP           DEFAULT CURRENT_TIMESTAMP
    -> );
Query OK, 0 rows affected (0.11 sec)
```

Fig. 8.9 User Table

8.3.2 Health Forms Table

The health_forms table stores one record per form submission. A single user can submit multiple forms over time, creating a history of their health data. The user_id column is a foreign key referencing the users table's id column with ON DELETE CASCADE, meaning all health form records for a user are automatically deleted when the user account is deleted. The health_issues column stores the comma-separated list of conditions selected by the user from the checkboxes as a VARCHAR(255) string.

```
mysql> CREATE TABLE IF NOT EXISTS health_forms (
    ->     id             INT AUTO_INCREMENT PRIMARY KEY,
    ->     user_id        INT                NOT NULL,
    ->     age            INT                NOT NULL,
    ->     gender         VARCHAR(10)        NOT NULL,
    ->     height_cm      FLOAT              NOT NULL,
    ->     weight_kg      FLOAT              NOT NULL,
    ->     bmi            FLOAT              NOT NULL,
    ->     water_intake   FLOAT              NOT NULL,
    ->     activity_level VARCHAR(20)        NOT NULL,
    ->     bp             INT                NOT NULL,
    ->     sugar          FLOAT              NOT NULL,
    ->     alcohol        TINYINT(1)         NOT NULL DEFAULT 0,
    ->     smoking        TINYINT(1)         NOT NULL DEFAULT 0,
    ->     health_issues  VARCHAR(255)       DEFAULT NULL,
    ->     submitted_at   TIMESTAMP          DEFAULT CURRENT_TIMESTAMP,
    ->
```

Fig. 8.10 health_formsTable

8.3.3 Diet Results Table

The diet_results table stores one record per ML prediction. The form_id column is a foreign key referencing the health_forms table's id column with ON DELETE CASCADE. The diet_plan_json column is a LONGTEXT type capable of storing the complete 30-day meal plan serialized as a JSON string, which can be several kilobytes in size. The predicted_condition column stores the condition name as a VARCHAR(50) string.

The three-table design with foreign keys creates a complete audit trail. By joining all three tables, an administrator can see exactly which health values a user entered, what condition was predicted, and what diet plan was generated — all linked by timestamp.

```
ysql> CREATE TABLE IF NOT EXISTS diet_results (
   ->     id                   INT AUTO_INCREMENT PRIMARY KEY,
   ->     user_id              INT             NOT NULL,
   ->     form_id              INT             NOT NULL,
   ->     predicted_condition  VARCHAR(50)     NOT NULL,
   ->     bmi                  FLOAT           NOT NULL,
   ->     diet_plan_json       LONGTEXT        NOT NULL,
   ->     generated_at         TIMESTAMP       DEFAULT CURRENT_TIMESTAMP,
   ->
   ->     FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE,
   ->     FOREIGN KEY (form_id) REFERENCES health_forms(id) ON DELETE CASCADE
   -> );
uery OK, 0 rows affected (0.12 sec)
```

Fig. 8.11 diet_results Table

8.4 Security Implementation

8.4.1 Password Security

All passwords are hashed using the bcrypt algorithm before being stored in the database. bcrypt is a password hashing function designed specifically for password storage. It is deliberately slow, making brute-force attacks computationally expensive. Each password hash includes a unique random salt generated by bcrypt.gensalt(), which means two users with the same password will have different hashes in the database. Even if the database is compromised, the original passwords cannot be recovered from the hashes.

8.4.2 SQL Injection Prevention

All database queries use parameterized placeholders. The mysql-connector-python library's cursor.execute() method accepts a tuple of parameters separately from the SQL string. The connector handles all escaping, quoting, and type conversion automatically. No user input is ever concatenated directly into a SQL string, completely eliminating the risk of SQL injection attacks.

8.4.3 Input Validation

All form inputs are validated on both the client side (JavaScript) and the server side (Flask). Client-side validation provides immediate user feedback without a round trip to the server. Server-side validation provides a security backstop in case the client-side validation is bypassed using browser developer tools or direct API calls. Flask wraps all route logic in try-except blocks and returns structured JSON error responses with appropriate HTTP status codes.

8.4.4 CORS Configuration

Flask-CORS is configured to allow requests from the frontend served by Flask itself. This prevents unauthorized API calls from external origins or malicious scripts running on other domains.

8.5 System Demonstration

8.5.1 Scenario A — New User Registration and Diabetes Plan

A 45-year-old male user opens the NutriMind website by running python app.py. The browser opens automatically at http://localhost:5000.The home page loads showing the 99.5% accuracy badge and the six covered conditions. The user clicks Login and then Create one to go to the signup page. He enters his name as Arjun Kumar, phone as 9876543210, email as arjun@gmail.com, and sets a password. The password strength meter shows Strong. He clicks Create Account. Flask inserts his record into the users table with a bcrypt hash. He is redirected to the health form.

On the health form, he enters age 45, gender Male, height 168 cm, weight 76 kg. BMI auto-calculates to 26.9 and shows the Overweight badge. He enters blood pressure 128 mmHg, blood sugar 178 mg/dL, water intake 2.0 litres, and selects Sedentary activity. He clicks No for smoking and No for alcohol. He leaves the health issues checkboxes unchecked. He clicks Get My Diet Plan.

Flask receives the data and constructs the feature array. Glucose value of 178 is significantly elevated.

The Random Forest model predicts Diabetes. Flask saves the form to health_forms and the result to diet_results. The result page loads showing the Diabetes Diet Plan badge and the complete 30-day meal plan. Day 1 shows Oats porridge with flaxseeds for breakfast, Brown rice with dal and salad for lunch, Handful of almonds for snacks, and Grilled fish with steamed vegetables for dinner. The user clicks Download PDF and saves NutriMind_Diabetes_DietPlan.pdf to his downloads folder.

8.5.2 Scenario B — Hypertension Prediction

A 55-year-old female user enters age 55, gender Female, height 160 cm, weight 72 kg giving BMI 28.1 shown as Overweight. She enters blood pressure 158 mmHg (significantly elevated), blood sugar 98 mg/dL (normal range), water intake 1.8 litres, and selects Light activity. She clicks Yes for smoking and No for alcohol. She leaves health issues unchecked.

The ML model receives blood pressure 158 as the dominant feature. The derived cholesterol proxy from the elevated BP also contributes. The Random Forest predicts Hypertension. The result page loads with the Hypertension Diet Plan badge. The diet plan shows low-sodium DASH-inspired meals. Day 1 shows Banana with oats porridge (low salt) for breakfast, Brown rice with dal (no extra salt) and salad for lunch, Watermelon slices for snacks, and Grilled fish with steamed vegetables for dinner.

8.5.3 Scenario C — Direct Condition Selection Bypassing ML

A user who has been formally diagnosed with Heart Disease by her cardiologist uses the system. She fills in all health vitals and then checks the Heart Disease checkbox in the Known Health Issues section. When she clicks Get My Diet Plan, Flask detects the selected condition in the health_issues list and directly maps it to the Heart Disease meal plan without calling model.predict(). The result page loads immediately with the Heart Disease Diet Plan badge showing omega-3 rich, low-cholesterol meals including oats with flaxseeds and walnuts for breakfast, brown rice with dal and salad (no oil) for lunch, almonds for snacks, and grilled fish with steamed vegetables for dinner.

8.5.4 Scenario D — Returning User Checking Previous Result

A user who previously used the system logs in again with their email and password. They can navigate to their previous result by fetching the /result/[email] Flask route which queries the diet_results table ordered by generated_at descending and returns the most recent result. The previous predicted condition, BMI, and complete diet plan are returned. This allows the user to review their past plan without resubmitting the health form.

8.6 System Performance

The system was tested for response time on a standard laptop with Intel Core i5 processor and 8 GB RAM running Windows 11.

Tab . 8.1 : System Performance Metrics

Operation	Average Time
Flask server startup (full load)	3.2 seconds
Browser auto-open after startup	4.7 seconds
Home page load time	0.8 seconds
Login authentication	0.4 seconds
Signup and redirect	0.5 seconds
ML prediction (/predict route)	1.1 seconds
Result page render (30 days)	0.6 seconds
PDF generation (30 days)	2.8 seconds
MySQL INSERT (health_forms)	0.3 seconds
MySQL INSERT (diet_results)	0.3 seconds

All critical operations complete within 3 seconds, meeting the performance target defined in the functional requirements. The ML prediction time of 1.1 seconds includes feature construction, model inference across 300 trees, meal plan lookup, and two MySQL INSERT operations.

8.7 Overcoming Implementation Challenges

8.7.1 Windows Path Error in Training Script

During initial development on a Windows machine, running train_final_model.py produced a SyntaxError related to Unicode escape sequences in file paths. The backslash character in Windows paths such as C:\Users\harip\datasets was being interpreted as an escape sequence by Python. This was resolved by replacing all hardcoded paths with os.path.join() and os.path.dirname(os.path.abspath(file)) which constructs platform-independent paths automatically at runtime.

8.7.2 Low Initial Model Accuracy

The first version of the ML model trained on the original Kaggle diet recommendations dataset achieved only 31.5% accuracy because all six conditions had almost identical feature value distributions. Blood pressure, glucose, and BMI values overlapped heavily across conditions in that dataset. This was resolved by rebuilding the training dataset from scratch using three separate Kaggle datasets and engineering clearly distinct feature ranges for each condition — Hypertension records always have BP above 140, Diabetes records always have glucose above 140, and Obesity records always have BMI above 30 with sedentary activity. This redesign increased accuracy from 31.5% to 99.50%.

8.7.3 joblib Warning Messages During Training

When running the training script on Python 3.14 on Windows, numerous warning messages from the joblib library's resource tracker appeared in the terminal. These warnings were caused by a compatibility issue between Python 3.14 and joblib's internal memory-mapped folder cleanup mechanism. The warnings did not affect model training or file saving. The solution was to ignore the warnings since the final output lines confirmed that all model files were saved successfully with correct accuracy values.

8.7.4 Flask Serving Frontend Pages

Initially, the frontend HTML pages could not be opened directly through the browser using the localhost URL because Flask did not have routes for them. This was resolved by adding a catch-all route in Flask that serves files from the frontend directory using Flask's send_from_directory() function. A home route for "/" serves index.html directly. A dynamic route for "/path:filename" serves any other file with a dot in the name from the frontend folder.

8.7.5 PDF Page Number Implementation

The jsPDF library's PageNumber class was not a constructor in the installed version, causing a TypeError when attempting to add page numbers to the PDF. This was resolved by using the SimpleField("PAGE") approach within a TextRun to insert a dynamic page number field. After further testing, the final implementation used jsPDF's internal getNumberOfPages() method in a post-processing loop that iterates over all pages after the document is complete and adds the page number text manually using doc.text() with the current page number and total page count.

CHAPTER 9

SUMMARY

9.1 Summary of Findings

This dissertation has presented the design, development, and evaluation of NutriMind — a Diet Plan Recommendation System for Health Issues Using Machine Learning. The project successfully achieved all its primary and sub-objectives.

The key finding is that Machine Learning can be applied effectively to automated dietary recommendation when the training dataset has clearly distinct feature ranges per condition. The Random Forest Classifier achieved 99.50% test accuracy and 99.56% five-fold cross-validation accuracy on an 18,000-record merged dataset covering six health conditions.

The feature importance analysis confirmed that cholesterol (33.45%), blood pressure (23.92%), and glucose (18.48%) are the three most predictive features — all of which are primary clinical indicators for the conditions they predict. This validates that the model has learned medically meaningful patterns.

The full-stack integration of Python Flask, MySQL, and HTML/CSS/JS demonstrated that a production-quality health application can be built using entirely open-source tools with zero licensing cost. The system provides real business value by delivering personalized dietary guidance that would otherwise require expensive professional consultation.

9.2 Recommendations

For individuals using the system: The dietary plans provided are based on established Medical Nutrition Therapy principles. Users should treat the recommendations as evidence-based guidance and not as a replacement for clinical consultation, particularly for severe health conditions.

For educational institutions: NutriMind demonstrates how a final-year project can combine multiple domains — Machine Learning, Web Development, and Healthcare — into one integrated system. Similar multi-domain projects are recommended as they develop cross-functional skills.

For future developers: Cloud deployment using AWS, Render, or Railway would make the system accessible to the public without requiring local installation. Adding a direct cholesterol input field and a real Thyroid dataset would further improve accuracy.

9.3 Future Work

The following enhancements are planned for future versions:

- Mobile Application: A React Native or Flutter mobile app with push notifications for daily meal reminders would improve user engagement significantly.
- Calorie Tracking: Adding calorie and macronutrient information (protein, carbohydrates, fat) for each meal would make the system more clinically comprehensive.
- Doctor Integration: Allowing registered dieticians to review and customize AI-generated plans would add a human oversight layer.

- Multilingual Support: Adding Tamil, Hindi, and other Indian regional language support would increase accessibility for non-English speakers.
- XGBoost Algorithm: Adding XGBoost as a fourth algorithm for comparison would strengthen the algorithmic analysis section.
- Wearable Integration: Connecting with fitness trackers to obtain real- time activity and heart rate data would improve prediction accuracy without requiring manual data entry.

CHAPTER 10
REFERENCES

1. Müller, M., & Kersten, W. (2017). Nutritional advisory systems using ML classifiers. International Journal of Medical Informatics, 107, 23–31.

2. Alghamdi, A. (2018). Predictive analytics for healthcare diet recommendations. IEEE Access, 6, 55057–55064.

3. Boag, W., & Weng, W. H. (2018). Food and nutrition recommendation using deep learning. IEEE Transactions on Multimedia, 20(8), 1926–1934.

4. Raj, S., & Rajesh, R. (2019). A personalized diet recommendation system using machine learning. International Journal of Computer Applications, 178(41), 22–27.

5. Bhatia, M., & Sood, M. (2020). Health monitoring and diet recommendation using data science. IEEE International Conference on Computing and Communication Systems.

6. Deepa, K., & Ramesh, S. (2020). A survey of machine learning techniques for nutrition recommendation. Elsevier Information Processing in Agriculture, 7(3), 447–460.

7. Shinde, S., & Kulkarni, A. (2020). Diet planning using machine learning-based classification. Elsevier Expert Systems with Applications, 162, 113847.

8. Sharma, P., & Aggarwal, R. (2021). Machine learning-based nutrition recommendation system. Elsevier Journal of King Saud University - Computer and Information Sciences.

9.Singh, A., & Gupta, V. (2021). Smart healthcare recommendation using machine learning. Elsevier Computers in Biology and Medicine, 133, 104400.

10.Fatima, M., & Ghoneim, A. (2022). Personalized healthcare prediction using ML algorithms. IEEE Journal of Biomedical and Health Informatics, 26(5), 2040–2050.

APPENDIX A

SAMPLE CODE AND LOGIC

A.1 Model Training — Random Forest (train_final_model.py)

```
model = RandomForestClassifier(
        n_estimators=300,
        max_depth=20,
        min_samples_split=3,
        random_state=42,
        n_jobs=-1
)
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
acc = accuracy_score(y_test, y_pred)
print(f"Test Accuracy: {acc*100:.2f}%")
# Output: Test Accuracy: 99.50%
```

A.2 Flask Predict Route (app.py)

```
@app.route("/predict", methods=["POST"])
def predict():
    data = request.json
    features = np.array([[age, bmi, bp, sugar, cholesterol, smoking, alcohol, activity_enc,
gender_enc]])
    pred = model.predict(features)
    condition = le.inverse_transform(pred)[0]
    diet_plan = meal_plans.get(condition, meal_plans["Normal"])
      return jsonify({
            "status": "success",
            "predicted_condition": condition,
            "diet_plan": diet_plan
    })
```

A.3 MySQL Database Setup (database_setup.sql)

```
CREATE DATABASE nutrimind_db;
USE nutrimind_db;


CREATE TABLE users (
id INT AUTO_INCREMENT PRIMARY KEY,
name VARCHAR(100) NOT NULL,
phone VARCHAR(15) NOT NULL,
email VARCHAR(150) UNIQUE NOT NULL,
password_hash VARCHAR(255) NOT NULL,
created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);


CREATE TABLE health_forms (
id INT AUTO_INCREMENT PRIMARY KEY,
 user_id INT NOT NULL,
age INT NOT NULL,
 bmi FLOAT NOT NULL,
bp INT NOT NULL,
sugar FLOAT NOT NULL,
submitted_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
 FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE
 );


CREATE TABLE diet_results (
id INT AUTO_INCREMENT PRIMARY KEY,
user_id INT NOT NULL,
form_id INT NOT NULL,
predicted_condition VARCHAR(50) NOT NULL,
```

```
diet_plan_json LONGTEXT NOT NULL,
generated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
 FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE,
FOREIGN KEY (form_id) REFERENCES health_forms(id) ON DELETE CASCADE
);
```